周报 W1 ｜ 吴源泰 ｜ 2026-10-07 ~ 2026-10-10

Phase 0（环境）· 项目根 `F:\research` · 主机 laptop-csp8hpp8 · 实际投入约 15.5 h

0. 一句话结论

Phase 0 中**不依赖数据集的部分已全部完成且留有可复现证据**（两个 conda 环境 + GPU 冒烟、源码按 commit 锚定、显存安全上限实测、手算边界例子、距离数值契约）；**唯一系统性阻塞是 MVTec AD / AD 2 官网登记未获批**，依赖真实图像的检查（含 PatchCore 最小示例）顺延到数据到位后第一时间补做。细节见 `week1\W1_EVIDENCE.md`。

1. Done ｜ 合计 15.5 h

任务	h	产物	状态
A14-1 硬件/数据清单 + README	2.5	`week1\environment.md`	✅
Phase 0 环境隔离（两个 conda 环境 + GPU 冒烟）	4.0	`logs\env_*.json`、`pip_freeze_*.txt`	✅
目录骨架补齐 + 源码锚定	1.5	`logs\anchors_latest.json`	✅
A14-3 六向量/两中心手算 `L, s_M, U`（含阈值并列与升级）	2.0	`project\src\w1_bounds_example.py`	✅
A14-2 回退：十个人造二维向量距离检查 + 精确距离核对	3.0	`project\src\w1_distance_checks.py`	✅
显存安全上限与功耗档实测	1.5	`logs\w1_vram_limit.json`	✅
A14-4 三角色清单生成器（含合成自检）	2.0	`scripts\make_manifests.py`	⚠ 真实清单待数据
A14-2 PatchCore 最小示例（真实图像）	0	—	⛔ 数据未获批

2. Evidence

主表：实测显存安全上限（判据：峰值 ≤ 80% 物理显存；骨干 WRN-50 冻结、常驻 265 MiB；两轮一致）

分辨率	安全 batch	峰值显存	占 8187 MiB	溢出起点（峰值 > 100%，且不报 OOM）
256×256	256	3787 MiB	46.3%	未观测到
384×384	192	6215 MiB	75.9%	256（100.1%）
512×512	112	6435 MiB	78.6%	160（110.8%）

- **配置/锚点**：env `adedge-patchcore`（py3.10.22、torch 2.6.0+cu124、freeze sha256 `70612E24…`）；patchcore-inspection `fcaa92f1…`（树指纹 `419fbc57…`）、AnomalyDINO `b9d1c264…`（`d23f5352…`）
- **数据划分**：真实清单未生成；划分逻辑经合成自检通过（16/24/60，**跨角色重叠 0**，两次运行逐字节一致）
- **数值契约**：分块与暴力精确最近邻最大差 **0.00e+00**；**GPU float32 vs CPU = 2.38e-07**（比 float64 的 1e-9 容差大两个数量级 → 保护量须按计算路径取）
- **日志**：`logs\anchors_latest.json`、`logs\env_adedge-patchcore_20261010-102504.json`、`logs\w1_distance_checks.log`、`logs\w1_vram_limit.json`
- **复现**：`powershell -ExecutionPolicy Bypass -File F:\research\scripts\check_env.ps1 -Python "C:\Users\yuantai\.conda\envs\adedge-patchcore\python.exe" -EnvName adedge-patchcore -Root F:\research`

- **代码推送**：`git@github.com:CAPOOCATXP/research01.git`（branch `main`，首次推送成功；SSH 专用密钥 `~/.ssh/id_ed25519_github`）

3. Blockers

1. **数据获取受阻（本周最主要问题）**：官网为 "Fill in the form"；官方评测码可直链下载且已获取，**未绕开登记用第三方镜像**。需要：请导师以学校邮箱登记，或明确授权使用某个具体镜像。

已核实的事实：MVTec AD 官方页面写明数据以 **CC BY-NC-SA 4.0** 发布——该许可**允许再分发**（署名 + 非商用 + 相同方式共享），因此第三方镜像在法律上可用；但问题在于 ① 偏离导师指定的"官方来源"，

② 镜像可能被裁剪，导致 bottle/tile/screw 的样本数与划分**与文献不可比**，"≥2 个百分点"的判据随之失效。

本周决定：暂不采用镜像（用户 2026-10-10 明确"暂时不做"）。

排期影响：Phase 2 的复现、按划分列图像与标签、PatchCore 最小示例、AnomalyDINO 冒烟**全部顺延**；

MVTec AD 2 带私有测试服务器、不存在可用镜像，必须走官方渠道。关键路径目前就卡在这一步。

2. **`git clone` 在本机阻塞**（`ls-remote` 正常，pack 传输 5 分钟无输出；已试 5 种参数组合）。已改用 ZIP 快照 + commit SHA + 树指纹并写入方法学。需要：认可该方式或提供可用代理。
3. **gh CLI 未安装 → 已解决**：改用 SSH 专用密钥 `~/.ssh/id_ed25519_github` 直接推送，`main` 分支已上传到 `github.com/CAPOOCATXP/research01`。`gh` 仍然缺失，但本阶段不需要它。

4. Next（Week 2 前三件事）

1. 数据到手后立即生成真实三角色清单并复查互斥 → 9 个 CSV + 计数表（2 h）
2. 补做被阻塞检查：列图像/标签、显示图像与掩码、跑一个 batch、PatchCore 最小文档示例、AnomalyDINO 1-shot 冒烟 → 配置、分数、patch map、逐种子日志（6 h）
3. 补录权重 sha256（WRN-50 / DINOv2 ViT-S/14）并落地 `run.json` 模板（2 h）

5. 最重要的负面结果

"不 OOM"不等于"没溢出"：原测法取"OOM 前一位 batch"，实测 512×512 下峰值达 **10835 MiB**，而 GPU 只有

8187 MiB —— Windows 驱动允许显存静默溢出到主机内存，不报错、只把时延拖慢约 20 倍（71.7 s vs 3.7 s）。

原测法作废，改为"安全区 + 溢出警戒"双判据。另两条：`git clone` 本机不可用；整块物化 `(512, 20000, 384)` 的 float64 距离矩阵需 **29.3 GiB**，第一版检查脚本因此直接内存溢出 —— **分块是必需，不是优化**。

并且块大小必须按内存预算反推：我曾写死 bs=256（单块中间张量 14.6 GiB），第一次靠内存空闲"跑通"、第二次同一条命令直接 `ArrayMemoryError` —— **一次跑通不等于可复现**，该结论已作废，改为按 2 GiB 预算自动定块。

6. 假设变更

原假设"显存上限 = 不 OOM 的最大 batch"被实测否定，已改为"物理显存内安全上限 + 溢出警戒线"双判据。

研究假设（压缩是否改变固定阈值下的**决策保持**）本周未触及，**无变更**。